

- 1) traer el repositorio: git clone <https://github.com/Lucasalexisc/sistemas-operativos>
- 2) cd sistemas-operativos
ls -la
- 3) Si hago cambios en la pc local, traigo los archivos con
git pull origin main
- 4) Para ver las carpetas
ls -l
- 5) cd Unidad_2
- 6) chmod +x demostracion_tpo.sh -> para dar permisos al archivo

Una vez que estoy parado en Unidad_2, tengo 4 archivos

carrera.c: Pone a 8 hilos a sumar 1 en una misma variable en la RAM 250000 veces. Deberia dar $8 * 250000 = 2000000$. Al no esperar entre hilos para hacer el proceso, se empiezan a pisar entre si. (falla de concurrencia)

seguro.c: es el mismo codigo que **carrera.c** pero con un **mutex**. El hilo que llega primero traba la puerta **lock**, suma 1 y al salir destraba con **unlock** para que pase el siguiente.

demostracion_tpo.sh: Mide el tiempo que tarda la CPu con y sin sincronizacion con el comando **time**.

Al correr demostracion_tpo.sh deberia devolver algo asi

```
=====
2. EJECUCION SIN CONTROL (CONDICION DE CARRERA)
=====
```

```
[RACE CONDITION] Total esperado: 2000000 | Total obtenido: 569399
```

```
=====
3. EJECUCION CON EXCLUSION MUTUA (MUTEX)
=====
```

```
[MUTEX LOCK] Total esperado: 2000000 | Total obtenido: 2000000
```

```
=====
4. COMPARACION DE TIEMPOS Y SOBRECARGA (OVERHEAD)
=====
```

```
--- Medición Sin Mutex ---
```

```
real 0m0.008s
user 0m0.021s
sys 0m0.002s
```

```
--- Medición Con Mutex ---
```

```
real 0m0.166s
user 0m0.174s
sys 0m0.392s
```

Que significa?

1) Perdida de datos 569399 vs 2000000

Hay una diferencia de ~1400000 operaciones- Por que pasa esto? en C se escribe como **contador_compartido ++**, pero la cpu no lo hace en un solo paso, ejecuta tres instrucciones:

- **MOV/LOAD:** leer el valor actual de la RAM y cargarlo en un registro del procesador
- **ADD:** Le suma 1 dentro de la ALU
- **MOV/STORE:** guarda el nuevo valor en la RAM

2) La solucion con **Mutex**

pthread_mutex_lock y **pthread_mutex_unlock** delimitan la **Sección Crítica**.

Solo permiten que un hilo a la vez ejecute ese tramo. Quien no tiene el cerrojo queda retenido esperando su turno, logrando la **Exclusión Mutua**.

3) Tiempos

real: Es el tiempo del reloj que pasa de 0.008s a 0.166. El programa tarda aprox

~20 veces mas porque las sumas se ordenaron en lugar de ejecutarse todas a la vez

user: Es el tiempo en espacio de usuario, es decir el tiempo total que gastaron los nucleos de la CPU procesando el codigo del programa con los **bucles for.**}

sys: Tiempo de espacio de kernel, cuando el hilo choca con una puerta que esta siendo ocupada por otro, no queda esperando en **espera activa**. Se hace una llamada al sistema con **futex** que pone al hilo a dormir en la cola de bloqueados, obligando a intervenir al nucleo, guardar su estado en su bloque de control y despertar a otro, lo que genera sobrecarga **overhead**.

Script carrera_escritura.sh

Que hace?

crea un archivo de texto llamado '**recurso_compartido.txt**', con la palabra VALOR_INICIAL. Luego arma dos funciones que compiten por escribir su propio nombre dentro del mismo archivo, simulando que dos procesos acceden al mismo recurso sin exclusion mutua

A. El operador **&** (Concurrencia en segundo plano)

- **¿Qué hace?:** Al poner **funcion_proceso1 &**, el shell no se queda esperando a que termine la función. Realiza una llamada al sistema tipo **fork()** en el kernel de Linux y lanza esa rutina como un **proceso hijo independiente** en segundo plano (*background*).
- **Impacto en el planificador:** Ahora hay dos procesos listos en memoria compitiendo al mismo tiempo por la CPU. Si el equipo tiene múltiples núcleos, el despachador (*dispatcher*) puede asignarlos físicamente en paralelo a núcleos distintos

B. La variable especial \$! y el comando wait

- **PID1=\$!:** La variable interna \$! de Bash guarda automáticamente el PID (*Process ID*) del último proceso lanzado en segundo plano.
- **wait \$PID1 \$PID2:** Es la instrucción de sincronización a nivel de procesos. Funciona igual que la llamada al sistema `wait()` o `waitpid()`: le dice al script principal que se quede en estado *Bloqueado/En espera* hasta que ambos procesos hijos finalicen su tarea, evitando leer el archivo antes de tiempo.

C. La redirección > (El recurso crítico no protegido)

- **¿Qué hace?:** `echo "PROCESO_1" > "$ARCHIVO_COMPARTIDO"` abre el archivo en modo truncado (lo vacía por completo y escribe la línea nueva desde el byte cero).
- **La condición de carrera:** El archivo de texto representa una **Sección Crítica** (un recurso compartido sin exclusión mutua. Como no hay ningún mecanismo de cerrojo o semáforo que cierre el acceso, los dos procesos entran al archivo cuando quieren).

D. La frase de cátedra: "El perdedor de la carrera determina el valor"

- **¿Quién pierde la carrera temporal?:** El **Proceso 1** tardó **0.05** segundos y escribió primero **PROCESO_1**. El **Proceso 2** fue más lento (tardó **0.08** segundos) y llegó después.
- **El resultado final:** El **Proceso 2** llegó segundo, abrió el archivo, borró lo que había hecho el **Proceso 1** y grabó **PROCESO_2**. El trabajo del primer proceso quedó totalmente descartado y destruido. Esto ilustra la definición exacta de Stallings y Tanenbaum: en una condición de carrera sin exclusión mutua, el último que escribe (el más lento en llegar) es el que impone el valor definitivo que queda en el sistema.